Redes convolucionales

Tomas Santana
November, 2026

Índice

1. Parte 1: Repaso: backpropagation 1
   1.1. La regla de la cadena 1
   1.2. `backward()` en PyTorch 2
2. Parte 2: Redes como clases de PyTorch 3
   2.1. `nn.Module`: `__init__` y `forward` 3
   2.2. Entrenar por mini-batches: épocas, mini-batches y pasos 4
   2.3. Evaluar el modelo 4
3. Parte 3: Redes convolucionales 5
   3.1. El problema de las imágenes 5
   3.2. La convolución 6
   3.3. Qué detecta un filtro 8
   3.4. Varios filtros y canales 8
   3.5. Padding y stride 10
   3.6. Max pooling 11
   3.7. Una red convolucional completa 12
   3.8. Por qué funcionan mejor con imágenes 14
   3.9. (Opcional) Reglas prácticas para diseñar una CNN 15
4. Parte 4: Sobreajuste y regularización 16
   4.1. Las curvas de entrenamiento y validación 16
   4.2. Dropout 16
   4.3. Weight decay y early stopping 17

1. Parte 1: Repaso: backpropagation

1.1. La regla de la cadena

Recordemos la red más pequeña que se describió anteriormente: una unidad oculta con ReLU y una salida sin activación,

$$z = w_1 x, \qquad a = \mathrm{ReLU}(z), \qquad \hat{y} = w_2 a, \qquad L = (\hat{y} - y)^2,$$

con $x = 2$, $y = 10$, $w_1 = 1$ y $w_2 = 1$. El forward pass calcula los valores de izquierda a derecha: $z = 2$, $a = 2$, $\hat{y} = 2$ y $L = 64$ (Figura 1, arriba).

Para el descenso por gradiente se necesitan $\frac{\partial L}{\partial w_1}$ y $\frac{\partial L}{\partial w_2}$. Backpropagation las calcula de derecha a izquierda (Figura 1, abajo). Se empieza con la derivada de la pérdida respecto a la predicción, $\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y) = -16$, a la que llamamos la señal de error. Después, en cada paso hacia atrás aplicamos la regla de la cadena:

- **Al cruzar una conexión**, la señal se multiplica por su peso: $\frac{\partial L}{\partial a} = \frac{\partial L}{\partial \hat{y}} \cdot w_2 = -16$.
- **Al cruzar una activación**, se multiplica por su pendiente. La ReLU tiene pendiente 1 si $z > 0$ y 0 si $z < 0$; aquí $z = 2$, así que $\frac{\partial L}{\partial z} = -16 \cdot 1 = -16$.
- **La derivada de un peso** es la señal que llega multiplicada por la entrada que recibe ese peso: $\frac{\partial L}{\partial w_2} = -16 \cdot a = -32$ y $\frac{\partial L}{\partial w_1} = -16 \cdot x = -32$.

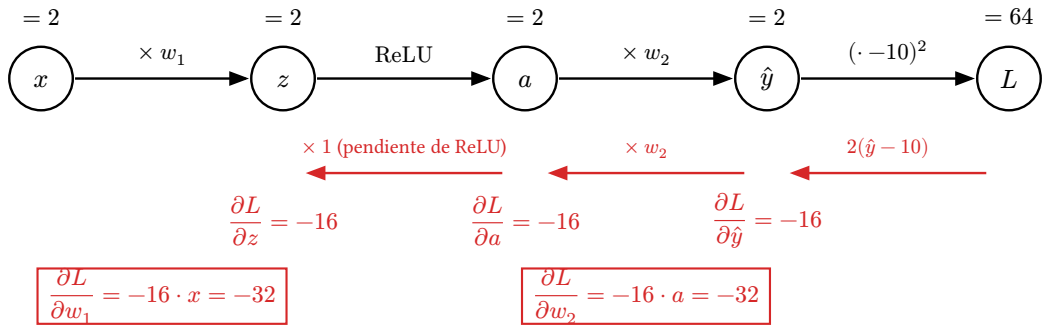


Figura 1: El cálculo de la red de dos pesos como una cadena de operaciones. Arriba, en negro, el forward pass. Abajo, en rojo, backpropagation: la derivada de la pérdida respecto a cada valor se calcula a partir de la del valor siguiente.

Con una red de millones de pesos el procedimiento es el mismo: un forward pass guarda los valores intermedios, y un recorrido hacia atrás reutiliza la señal de cada capa para calcular la de la capa anterior.

Ejercicio 1

Para la misma red, con $x = 1$, $y = 4$, $w_1 = 2$ y $w_2 = 3$:

1. Calcular el forward pass: z, a, $\hat{y}$ y L.
2. Calcular, de derecha a izquierda, $\frac{\partial L}{\partial \hat{y}}$, $\frac{\partial L}{\partial w_2}$, $\frac{\partial L}{\partial a}$, $\frac{\partial L}{\partial z}$ y $\frac{\partial L}{\partial w_1}$.
3. Dar un paso de descenso por gradiente con $\alpha = 0.01$. ¿Los pesos suben o bajan? ¿Por qué tiene sentido, viendo $\hat{y}$ y y?

1.2. `backward()` en PyTorch

PyTorch hace exactamente este cálculo. Al crear un tensor con `requires_grad=True`, PyTorch registra las operaciones del forward pass; al llamar `L.backward()`, recorre esas operaciones hacia atrás y guarda en `.grad` la derivada de `L` respecto a cada tensor.

```python
import torch

x, y = 2.0, 10.0
w1 = torch.tensor(1.0, requires_grad=True)
w2 = torch.tensor(1.0, requires_grad=True)

z = w1 * x                # forward pass
a = torch.relu(z)
y_hat = w2 * a
L = (y_hat - y) ** 2

L.backward()              # backpropagation
print("L =", L.item())
print("dL/dw1 =", w1.grad.item())
print("dL/dw2 =", w2.grad.item())
```

Salida

```
L = 64.0
dL/dw1 = -32.0
dL/dw2 = -32.0
```

2. Parte 2: Redes como clases de PyTorch

2.1. `nn.Module`: `__init__` y `forward`

Anteriormente se definieron redes utilizando `nn.Sequential`, que aplica una lista de capas en orden. Es cómodo para redes simples, pero en la práctica las redes se definen como una clase que extiende `nn.Module`. Esta clase tiene dos métodos especiales:

- **`__init__` declara las capas que tienen parámetros.** Cada capa se guarda como un atributo de `self` (por ejemplo, `self.hidden`). Así PyTorch las encuentra: `model.parameters()` recorre los pesos y sesgos de todas las capas guardadas en `self`, y eso es lo que le pasamos al optimizador. Lo primero que hace `__init__` es llamar `super().__init__()`, que prepara la clase para registrar esas capas.
- **`forward` describe el cálculo:** qué le pasa a la entrada `x`, en qué orden, hasta llegar a la salida. Aquí pueden aparecer también operaciones sin parámetros, como la ReLU o `flatten`, que no hace falta declarar en `__init__`.

Por ejemplo, esta es una red con una capa oculta para clasificar los dígitos de MNIST:

Python

```python
import torch
from torch import nn


class MyMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(784, 128)
        self.out = nn.Linear(128, 10)

    def forward(self, x):
        x = x.flatten(start_dim=1)  # (N, 1, 28, 28) -> (N, 784)
        x = torch.relu(self.hidden(x))  # (N, 128)
        return self.out(x)  # (N, 10): logits, sin softmax


if __name__ == "__main__":
    model = MyMLP()
    x = torch.rand(64, 1, 28, 28)  # un mini-batch de 64 imágenes
    print(model(x).shape)
    print(sum(p.numel() for p in model.parameters()))
```

Salida

```
torch.Size([64, 10])
101770
```

Algunos detalles:

- La entrada es un mini-batch de imágenes con forma $(N, 1, 28, 28)$: N imágenes, de 1 canal (escala de grises), de 28×28 píxeles. Una capa densa espera un vector por ejemplo, así que `forward` primero aplana cada imagen a un vector de 784 números. Con `start_dim=1` se aplanan todas las dimensiones excepto la primera, evitando mezclar los ejemplos del mini-batch.
- La salida tiene 10 números por imagen, uno por dígito. Son los scores (logits), sin softmax: como se vio anteriormente, `nn.CrossEntropyLoss` aplica softmax internamente.
- Para usar la red se escribe `model(x)`, no `model.forward(x)`.
- La red tiene $784 \cdot 128 + 128 + 128 \cdot 10 + 10 = 101770$ parámetros, como confirma la última línea.

2.2. Entrenar por mini-batches: épocas, mini-batches y pasos

Anteriormente se entrenó la red con todos los datos a la vez. Con 50000 imágenes eso es lento y ocupa mucha memoria, así que se usa descenso por gradiente con mini-batches: en cada paso, el gradiente se calcula con un grupo pequeño de ejemplos.

- Un **mini-batch** es un grupo pequeño de ejemplos de entrenamiento, por ejemplo 64.
- Un **paso** es una actualización de los parámetros: forward pass, pérdida, `backward()` y `step()` sobre un mini-batch.
- Una **época** es una pasada completa por los datos de entrenamiento: tantos pasos como mini--batches caben en los datos.

Por ejemplo, con 50000 ejemplos y mini-batches de 64, una época tiene $\lceil 50000/64 \rceil = 782$ pasos (el último mini-batch tiene solo 16 ejemplos). En PyTorch, un `DataLoader` se encarga de mezclar los datos en cada época y entregarlos en mini-batches:

```python
def train(model, X_train, y_train, X_val, y_val, num_epochs=3, lr=0.1):
    datos = DataLoader(TensorDataset(X_train, y_train),
                       batch_size=64, shuffle=True)
    loss = nn.CrossEntropyLoss()
    optimizer = torch.optim.SGD(model.parameters(), lr=lr)
    for epoca in range(num_epochs):
        model.train()               # modo entrenamiento
        for X_batch, y_batch in datos:  # un paso por mini-batch
            optimizer.zero_grad()
            L = loss(model(X_batch), y_batch)
            L.backward()
            optimizer.step()
        acc = accuracy(model, X_val, y_val)
        print(f"época {epoca + 1}: validación {acc:.3f}")
```

Ejercicio 2

1. ¿Cuántos pasos da `entrenar` con 50000 ejemplos, mini-batches de 64 y 3 épocas?
2. ¿Y si los mini-batches son de 128? ¿Cuántas veces ve la red cada ejemplo en los dos casos?

2.3. Evaluar el modelo

Para medir la exactitud se utiliza esta función:

```python
def accuracy(model, X, y):
    model.eval()              # modo evaluación
    with torch.no_grad():      # sin registrar operaciones
        predicted_classes = model(X).argmax(dim=1)
    return (predicted_classes == y).float().mean().item()
```

La clase predicha es la de mayor score (`argmax`). Hay dos detalles nuevos. `torch.no_grad()` le dice a PyTorch que no registre las operaciones, porque no se llamará a `backward()`; así la evaluación es más rápida. Y `model.eval()` pone la red en modo evaluación, mientras que `model.train()`, al principio de cada época, la pone en modo entrenamiento. Por ahora no cambia nada, pero algunas capas se comportan distinto al entrenar y al evaluar; lo que se verá en la Parte 4.

Entrenando `MyMLP` sobre 50000 imágenes de MNIST, con 10000 imágenes de validación, obtenemos:

> Salida
> ```
> época 1: validación 0.914
> época 2: validación 0.937
> época 3: validación 0.952
> ```

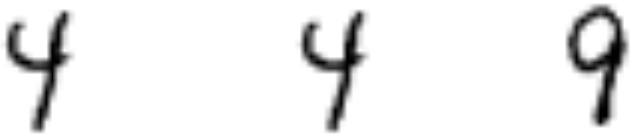
3. Parte 3: Redes convolucionales

3.1. El problema de las imágenes

La red `MyMLP` llega al 95.0% de exactitud en los datos de prueba. Funciona, pero tiene dos problemas que se agravan con imágenes más grandes.

- **Demasiados parámetros.** Cada unidad de la primera capa está conectada con todos los píxeles. Con imágenes de 28×28 en escala de grises son 784 pesos por unidad. Con una foto a color de 224×224 píxeles, la entrada tiene $224 \cdot 224 \cdot 3 = 150528$ números, y una capa oculta de 1000 unidades ya tiene más de 150 millones de pesos.
- **Ignora la estructura de la imagen.** Al aplanar la imagen, la red no sabe qué píxeles son vecinos: para ella, los 784 píxeles son 784 features sin ningún orden. Además, un mismo patrón en dos lugares distintos de la imagen es, para la red, una entrada completamente distinta. La Figura 2 lo muestra: si desplazamos un 4 cuatro píxeles a la derecha, la distancia entre las dos imágenes, vistas como vectores de 784 píxeles, es más del doble que la distancia del 4 original al 9 más parecido de los datos.

Esto significa que la red es muy sensible a pequeños desplazamientos o rotaciones.



un 4 | el mismo 4, desplazado 4 píxeles distancia al primero: **2 196** | el 9 más parecido al primer 4 distancia al primero: **1 027**

Figura 2: Para un modelo que trata la imagen como un vector de 784 píxeles, el 4 desplazado 4 píxeles está mucho más lejos del original (distancia euclidiana entre los vectores de píxeles) que el 9 más parecido de los datos de entrenamiento.

Las redes convolucionales (CNN) resuelven los dos problemas con una idea simple: en lugar de conectar cada unidad con toda la imagen, usan un filtro pequeño que recorre la imagen buscando el mismo patrón en todas las posiciones.

3.2. La convolución

Una convolución toma una imagen X de $n \times n$ y un filtro (o kernel) K de $k \times k$, más pequeño. El filtro se coloca sobre una ventana de $k \times k$ de la imagen, se multiplican los valores que quedan uno encima del otro y se suman los productos. Ese número es una entrada de la salida. Después el filtro se desplaza una posición y se repite, hasta recorrer toda la imagen (Figura 3). En fórmula, la entrada (i, j) de la salida es

$$y_{ij} = \sum_{u=1}^{k} \sum_{v=1}^{k} x_{i+u-1, j+v-1} K_{uv}.$$

Como el filtro cabe en $n - k + 1$ posiciones en cada dirección, la salida es de $(n - k + 1) \times (n - k + 1)$.

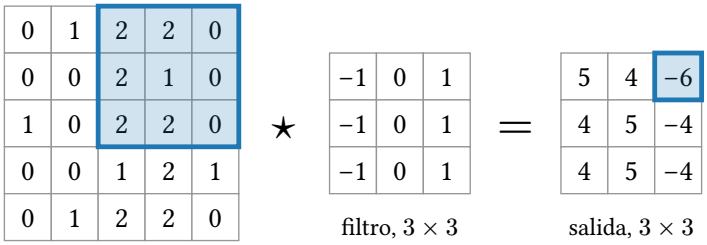


Figura 3: Una convolución de una entrada de 5×5 con un filtro de 3×3. La ventana resaltada produce la entrada resaltada de la salida: $2 \cdot (-1) + 2 \cdot 0 + 0 \cdot 1 + 2 \cdot (-1) + 1 \cdot 0 + 0 \cdot 1 + 2 \cdot (-1) + 2 \cdot 0 + 0 \cdot 1 = -6$.

Por ejemplo, la entrada de la esquina superior izquierda de la salida es

$$0 \cdot (-1) + 1 \cdot 0 + 2 \cdot 1 + 0 \cdot (-1) + 0 \cdot 0 + 2 \cdot 1 + 1 \cdot (-1) + 0 \cdot 0 + 2 \cdot 1 = 5.$$

En PyTorch, una capa convolucional es `nn.Conv2d`. Se pueden comprobar los cálculos de la figura con este código:

```python
import torch
from torch import nn

X = torch.tensor([[0., 1, 2, 2, 0],
                  [0, 0, 2, 1, 0],
                  [1, 0, 2, 2, 0],
                  [0, 0, 1, 2, 1],
                  [0, 1, 2, 2, 0]])
K = torch.tensor([[-1., 0, 1],
                  [-1, 0, 1],
                  [-1, 0, 1]])

conv = nn.Conv2d(1, 1, kernel_size=3, bias=False)
with torch.no_grad():
    conv.weight[0, 0] = K                # el filtro, a mano
    out = conv(X.reshape(1, 1, 5, 5))  # (N, C, H, W)

print(out.shape)
print(out[0, 0])
```

Salida

```
torch.Size([1, 1, 3, 3])
tensor([[ 5.,  4., -6.],
        [ 4.,  5., -4.],
        [ 4.,  5., -4.]])
```

PyTorch representa las imágenes como tensores de cuatro dimensiones, (N, C, H, W): el número de imágenes del mini-batch, el número de canales, el alto y el ancho. Nuestra imagen es un mini-batch de 1 imagen de 1 canal de 5×5, es decir, $(1, 1, 5, 5)$, y la salida es $(1, 1, 3, 3)$. La matriz de la salida es la misma de la figura.

Una aclaración sobre nombres. En matemáticas y en procesamiento de señales, la convolución voltea el filtro (lo gira 180°) antes de recorrer la imagen. Dentro de una CNN esto no es necesario (técnicalmente, a esto se le llama correlación cruzada). Como los filtros se aprenden, voltearlos no cambia nada, y nadie lo hace.

Ejercicio 3 (en papel)

Calcular la convolución de la entrada X con el filtro K:

$$X = \begin{pmatrix} 1 & 2 & 0 & 1 \\ 0 & 1 & 3 & 1 \\ 2 & 0 & 1 & 0 \\ 1 & 1 & 0 & 2 \end{pmatrix}, \qquad K = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

1. ¿De qué tamaño es la salida? Calcular sus entradas.
2. Aplicar la ReLU a la salida.
3. Si X fuera un tensor de PyTorch de forma $(1, 1, 4, 4)$, ¿qué forma tendría la salida de `nn.Conv2d(1, 1, kernel_size=2)`?

3.3. Qué detecta un filtro

¿Para qué sirve esta operación? Una entrada de la salida es grande cuando la ventana de la imagen se parece al filtro: los píxeles que coinciden con pesos positivos son altos y los que coinciden con pesos negativos son bajos. El filtro de la Figura 3 tiene -1 a la izquierda y $+1$ a la derecha, así que responde a ventanas con valores más altos a la derecha que a la izquierda: detecta **bordes verticales**. En la figura, la salida es positiva donde la franja de valores altos empieza y negativa donde termina.

La Figura 4 aplica dos filtros así a un dígito real de MNIST, seguidos de una ReLU: el de bordes verticales y uno de bordes horizontales. Cada salida es una imagen nueva, llamada **mapa de activación** (feature map), que muestra dónde aparece el patrón del filtro.

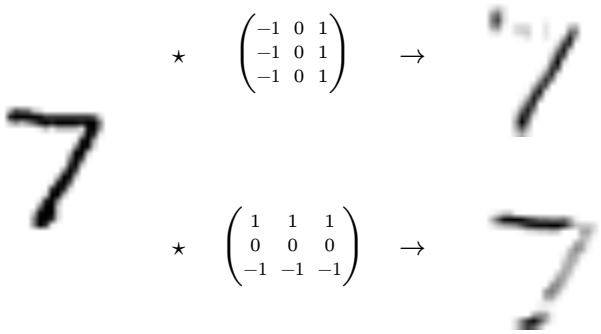


Figura 4: Dos filtros de 3×3 aplicados a un dígito de MNIST, seguidos de una ReLU. El primero detecta bordes verticales (el trazo empieza a la derecha de un fondo vacío) y el segundo, bordes horizontales (hay trazo arriba y no abajo). Las salidas son de 28×28 porque se agregó padding (sección 3.5).

Dentro de una CNN, los filtros se aprenden: los pesos se ajustan durante el entrenamiento para que cada filtro detecte un patrón útil para la tarea.

3.4. Varios filtros y canales

Un solo filtro detecta un solo patrón. Una capa convolucional usa varios filtros, cada uno con sus propios pesos, y produce un mapa de activación por filtro. Los mapas se apilan en un tensor: si la capa tiene 8 filtros, la salida tiene 8 **canales** (Figura 5). Además, cada filtro tiene un sesgo.

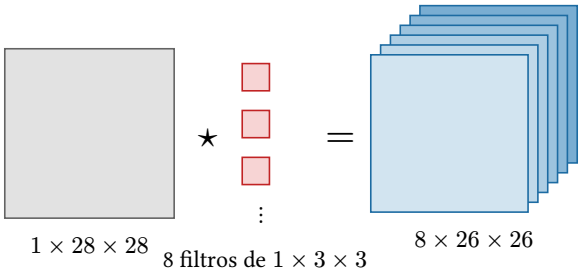


Figura 5: Una capa convolucional con 8 filtros de 3×3 sobre una imagen de 28×28 en escala de grises (1 canal). Cada filtro produce un mapa de 26×26, y la salida apila los 8 mapas.

```python
import torch
from torch import nn

x = torch.rand(64, 1, 28, 28)  # 64 imágenes de 1 canal

conv = nn.Conv2d(in_channels=1, out_channels=8, kernel_size=3)
print("pesos:", conv.weight.shape)  # 8 filtros de 1 x 3 x 3
print("sesgos:", conv.bias.shape)  # un sesgo por filtro
print("salida:", conv(x).shape)
print("parámetros:", sum(p.numel() for p in conv.parameters()))
```

Salida

```
pesos: torch.Size([8, 1, 3, 3])
sesgos: torch.Size([8])
salida: torch.Size([64, 8, 26, 26])
parámetros: 80
```

`nn.Conv2d(in_channels=1, out_channels=8, kernel_size=3)` crea 8 filtros. Sus pesos se guardan en un tensor de forma $(8, 1, 3, 3)$: 8 filtros de $1 \times 3 \times 3$. La salida de un mini-batch de 64 imágenes es $(64, 8, 26, 26)$.

Entradas con varios canales. ¿Qué pasa si la entrada ya tiene varios canales, por ejemplo los 8 mapas de la capa anterior, o los 3 canales (RGB) de una foto a color? Entonces cada filtro tiene un canal por cada canal de la entrada: es un bloque de $C_{\text{in}} \times k \times k$. El filtro se coloca sobre una ventana de $k \times k$ de **todos** los canales a la vez, multiplica y suma los $C_{\text{in}} k^2$ productos, y produce un solo número. Cada filtro sigue produciendo un solo mapa, sin importar cuántos canales tenga la entrada (Figura 6).

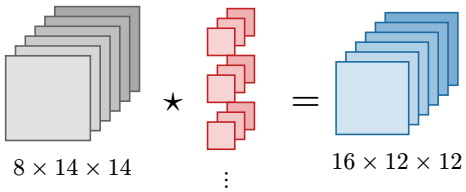


Figura 6: Una capa con 16 filtros sobre una entrada de 8 canales. Cada filtro es un bloque de $8 \times 3 \times 3$ que combina los 8 canales de la entrada en un solo mapa.

```python
import torch
from torch import nn

x = torch.rand(64, 8, 14, 14)  # 8 canales de entrada

conv = nn.Conv2d(in_channels=8, out_channels=16, kernel_size=3)
print("pesos:", conv.weight.shape)  # 16 filtros de 8 x 3 x 3
print("salida:", conv(x).shape)
print("parámetros:", sum(p.numel() for p in conv.parameters()))
```

Salida

```
pesos: torch.Size([16, 8, 3, 3])
salida: torch.Size([64, 16, 12, 12])
parámetros: 1168
```

Número de parámetros. Una capa con C_{in} canales de entrada, C_{out} filtros de $k \times k$ tiene C_{out} filtros de $C_{\text{in}}k^2$ pesos, más un sesgo por filtro:

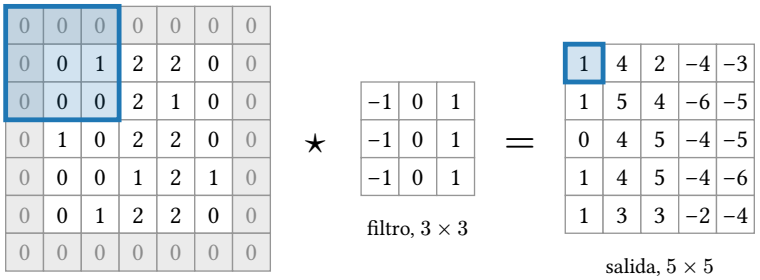
$$(C_{\text{in}}k^2 + 1)C_{\text{out}} \text{ parámetros.}$$

Notar que el número de parámetros **no depende del tamaño de la imagen**: el mismo filtro se usa en todas las posiciones.

3.5. Padding y stride

Cada convolución hace la imagen más pequeña: con un filtro de 3×3, una imagen de 28×28 pasa a 26×26. Además, los píxeles del borde participan en menos ventanas que los del centro. Para evitarlo se agrega **padding**: un borde de ceros alrededor de la entrada.

Por ejemplo, con un padding de 1 y un filtro de 3×3, la salida tiene el mismo tamaño que la entrada (Figura 7).



entrada, 7×7 (con padding) — filtro, 3×3 — salida, 5×5

Figura 7: La misma convolución de la Figura 3, con padding de 1: la entrada se rodea de ceros (en gris) y la salida queda de 5×5, igual que la entrada original.

El **stride** es cuántas posiciones avanza el filtro en cada paso. Con stride 1 recorre todas las posiciones; con stride 2 salta una de cada dos, y la salida queda de la mitad del tamaño.

En general, con una entrada de $n \times n$, un filtro de $k \times k$, padding p y stride s, la salida es de $m \times m$ con

$$m = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1.$$

Por ejemplo, para $n = 28$ y $k = 3$: sin padding, $m = 26$; con $p = 1$, $m = 28$; con $p = 1$ y $s = 2$, $m = \lfloor 27/2 \rfloor + 1 = 14$. Lo comprobamos:

```python
import torch
from torch import nn

x = torch.rand(1, 1, 28, 28)
for padding, stride in [(0, 1), (1, 1), (1, 2)]:
    conv = nn.Conv2d(1, 8, kernel_size=3, padding=padding, stride=stride)
    print(f"padding={padding}, stride={stride}:", tuple(conv(x).shape))
```

Salida

```
padding=0, stride=1: (1, 8, 26, 26)
padding=1, stride=1: (1, 8, 28, 28)
padding=1, stride=2: (1, 8, 14, 14)
```

3.6. Max pooling

Después de una convolución se suele agregar una capa de **pooling**, que tiene dos efectos útiles. Primero, reduce el tamaño de los mapas, y con eso el número de parámetros de las capas siguientes. Segundo, hace a la red un poco más tolerante a desplazamientos pequeños: si el filtro anterior detecta un borde, el pooling conserva esa información aunque pierda su posición exacta.

La más común es el **max pooling** de 2×2: divide cada mapa en bloques de 2×2 y se queda con el máximo de cada bloque (Figura 8). Así un mapa de 28×28 pasa a 14×14.

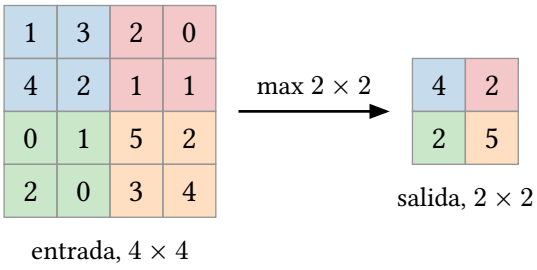


Figura 8: Max pooling de 2×2: cada bloque de color de la entrada se reduce a su máximo.

El pooling se aplica a cada canal por separado, así que no cambia el número de canales.

Python

```python
import torch
from torch import nn

X = torch.tensor([[1., 3, 2, 0],
                  [4, 2, 1, 1],
                  [0, 1, 5, 2],
                  [2, 0, 3, 4]])
pool = nn.MaxPool2d(kernel_size=2)
print(pool(X.reshape(1, 1, 4, 4))[0, 0])

x = torch.rand(64, 8, 28, 28)
print(x.shape, "->", pool(x).shape)
print("parámetros:", sum(p.numel() for p in pool.parameters()))
```

Salida

```
tensor([[4., 2.],
        [2., 5.]])
torch.Size([64, 8, 28, 28]) -> torch.Size([64, 8, 14, 14])
parámetros: 0
```

Ejercicio 4

1. Aplicar max pooling de 2×2 a la matriz

$$Z = \begin{pmatrix} 2 & 0 & 1 & 5 \\ 3 & 1 & 0 & 2 \\ 0 & 4 & 2 & 2 \\ 1 & 1 & 3 & 0 \end{pmatrix}.$$

2. Un tensor de forma $(64, 16, 14, 14)$ pasa por `nn.MaxPool2d(2)`. ¿Qué forma tiene la salida? ¿Cuántos parámetros tiene la capa?

3.7. Una red convolucional completa

Una CNN típica repite varias veces un **bloque** de convolución, ReLU y pooling. Cada bloque detecta patrones a partir de los mapas del bloque anterior: los primeros detectan bordes, y los siguientes combinan bordes en patrones más grandes. Mientras tanto, los mapas se hacen más pequeños y el número de canales crece. Al final, los mapas se aplanan en un vector y pasan por una o más capas densas, que hacen la clasificación.

La Figura 9 muestra la red que vamos a usar con MNIST, con dos bloques y una capa densa.

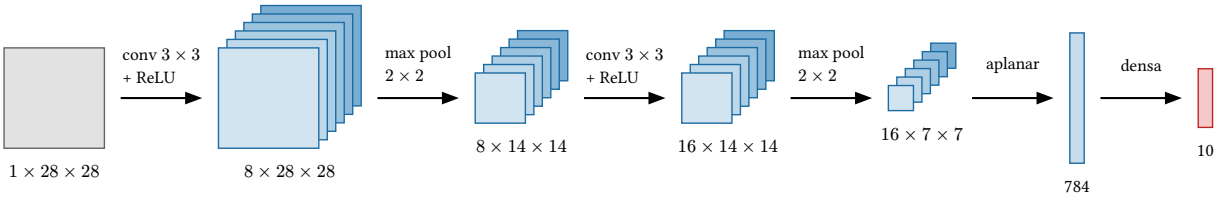


Figura 9: La arquitectura de `MyCNN`. Debajo de cada tensor, su forma (canales × alto × ancho), sin contar la dimensión del mini-batch.

```python
import torch
from torch import nn


class MyCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 8, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(8, 16, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2)
        self.out = nn.Linear(16 * 7 * 7, 10)

    def forward(self, x):
        x = self.pool(torch.relu(self.conv1(x)))   # (N, 8, 14, 14)
        x = self.pool(torch.relu(self.conv2(x)))   # (N, 16, 7, 7)
        x = x.flatten(start_dim=1)                 # (N, 784)
        return self.out(x)                        # (N, 10)
```

Como siempre, `__init__` declara las capas con parámetros y `forward` describe el cálculo. La misma capa `self.pool` se usa dos veces, porque no tiene parámetros. El número más difícil de este código

es el `16 * 7 * 7` de la capa densa: es el número de entradas que recibe después de aplanar, y hay que calcularlo siguiendo las formas desde la entrada:

Capa	Forma de la salida	Parámetros
entrada	$1 \times 28 \times 28$	—
conv1: `Conv2d(1, 8, 3, padding=1)` + ReLU	$8 \times 28 \times 28$	$(1 \cdot 9 + 1) \cdot 8 = 80$
pool: `MaxPool2d(2)`	$8 \times 14 \times 14$	0
conv2: `Conv2d(8, 16, 3, padding=1)` + ReLU	$16 \times 14 \times 14$	$(8 \cdot 9 + 1) \cdot 16 = 1168$
pool: `MaxPool2d(2)`	$16 \times 7 \times 7$	0
aplanar	784	0
densa: `Linear(784, 10)`	10	$784 \cdot 10 + 10 = 7850$
Total		**9098**

Si el número de entradas de la capa densa está mal, PyTorch da un error de dimensiones en el primer forward pass. La forma más segura de verificarlo es hacer un **forward de prueba**: pasar una imagen inventada por la red, capa por capa, e imprimir la forma después de cada una. Y para contar los parámetros de cada capa se usa `p.numel()`:

```python
import torch
from cnn import MyCNN

model = MyCNN()
x = torch.rand(1, 1, 28, 28)  # una imagen inventada
print("entrada", tuple(x.shape))
x = torch.relu(model.conv1(x))
print("conv1  ", tuple(x.shape))
x = model.pool(x)
print("pool   ", tuple(x.shape))
x = torch.relu(model.conv2(x))
print("conv2  ", tuple(x.shape))
x = model.pool(x)
print("pool   ", tuple(x.shape))
x = x.flatten(start_dim=1)
print("flatten", tuple(x.shape))
x = model.out(x)
print("densa  ", tuple(x.shape))

for nombre, p in model.named_parameters():
    print(f"{nombre:13} {str(tuple(p.shape)):15} {p.numel():5}")
```

Salida

```
entrada (1, 1, 28, 28)
conv1   (1, 8, 28, 28)
pool    (1, 8, 14, 14)
conv2   (1, 16, 14, 14)
pool    (1, 16, 7, 7)
flatten (1, 784)
densa   (1, 10)
conv1.weight  (8, 1, 3, 3)       72
conv1.bias    (8,)                8
conv2.weight  (16, 8, 3, 3)    1152
conv2.bias    (16,)              16
densa.weight  (10, 784)        7840
densa.bias    (10,)              10
```

Ejercicio 5

Considerar esta red para imágenes de $1 \times 28 \times 28$, sin padding: `Conv2d(1, 6, kernel_size=5)`, ReLU, `MaxPool2d(2)`, `Conv2d(6, 16, kernel_size=5)`, ReLU, `MaxPool2d(2)`, aplanar y `Linear(?, 10)`.

1. Calcular la forma de la salida de cada capa.
2. ¿Cuántas entradas tiene la capa densa?
3. ¿Cuántos parámetros tiene cada capa? ¿Y la red completa?

3.8. Por qué funcionan mejor con imágenes

Entrenando `MyCNN` con el mismo código y los mismos datos que `MyMLP`:

Salida

```
época 1: validación 0.957
época 2: validación 0.938
época 3: validación 0.976
```

Red	Parámetros	Exactitud de validación	Exactitud de prueba
`MyMLP`	101 770	95.2%	95.0%
`MyCNN`	9 098	97.6%	97.6%

La CNN es más precisa y tiene once veces menos parámetros. La razón está en dos propiedades de la convolución:

- **Conexiones locales.** Cada entrada de un mapa depende solo de una ventana pequeña de la imagen, no de todos los píxeles. Los patrones de una imagen (bordes, esquinas, trazos) son locales, así que no se pierde nada.
- **Pesos compartidos.** El mismo filtro se usa en todas las posiciones de la imagen. Un detector de bordes que funciona arriba a la izquierda también funciona abajo a la derecha.

La Figura 10 muestra los 8 filtros de la primera capa de `MyCNN` después de entrenar, y sus mapas de activación para un dígito de prueba. Varios se parecen a los detectores de bordes de la Figura 4.

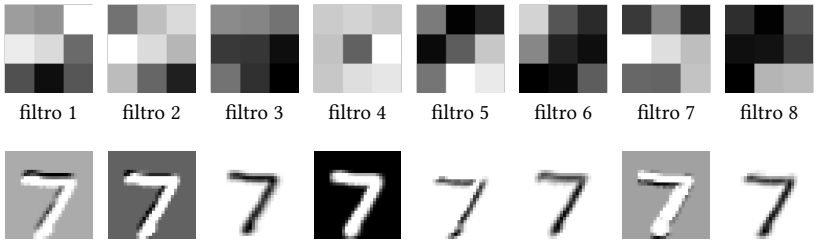


Figura 10: Arriba, los 8 filtros de 3×3 de la primera capa de `MyCNN` después de entrenar (oscuro: peso positivo; claro: negativo). Abajo, el mapa de activación de cada filtro (después de la ReLU) para un 7 de los datos de prueba.

3.9. (Opcional) Reglas prácticas para diseñar una CNN

¿Cuántas capas, cuántos filtros, de qué tamaño? No hay una fórmula que dé la mejor arquitectura para un problema, y en la práctica se prueban varias y se comparan con los datos de validación. Pero la mayoría de las CNN siguen el mismo patrón general:

$$\text{entrada} \to [[\text{conv} \to \text{ReLU}] \times N \to \text{pool}] \times M \to [\text{densa} \to \text{ReLU}] \times K \to \text{densa}.$$

Es decir, M bloques, cada uno con N convoluciones (con su ReLU) seguidas de un pooling, y al final K capas densas ocultas antes de la capa de salida. Lo usual es N entre 1 y 3 y K entre 0 y 2; M depende del tamaño de las imágenes. Nuestra `MyCNN` tiene $N = 1$, $M = 2$ y $K = 0$.

Dentro de ese patrón, estas reglas funcionan bien como punto de partida:

- **Filtros pequeños.** Usar filtros de 3×3 (a lo sumo 5×5) con stride 1. Para detectar patrones grandes es mejor apilar varias capas de filtros pequeños que usar un filtro grande: cada capa combina los patrones de la anterior, y entre capa y capa hay una ReLU.
- **Padding para conservar el tamaño.** Con un filtro de $k \times k$ y stride 1, un padding de $p = (k - 1)/2$ deja la salida del mismo tamaño que la entrada: $p = 1$ para $k = 3$ y $p = 2$ para $k = 5$ (se puede comprobar con la fórmula de la sección 3.5). Así las convoluciones no cambian el tamaño, solo el pooling lo hace, y es mucho más fácil seguir las formas.
- **Pooling de 2×2.** Cada pooling divide el alto y el ancho entre 2. Conviene que el tamaño de las imágenes se pueda dividir entre 2 varias veces. Con MNIST, dos poolings llevan de 28×28 a 7×7; un tercero daría 3×3, porque PyTorch descarta la última fila y la última columna cuando el tamaño es impar.
- **Más filtros a medida que la imagen se achica.** Lo usual es duplicar el número de filtros después de cada pooling, como en `MyCNN` (8 y luego 16). Los mapas son cada vez más pequeños, así que podemos pagar más canales, y las capas profundas necesitan más filtros porque combinan los patrones de las anteriores de muchas formas distintas.
- **Cuidado con la primera capa densa.** Casi todos los parámetros de una CNN suelen estar en la primera capa densa, que recibe todos los mapas aplanados. En `MyCNN`, 7850 de los 9098 parámetros (el 86%) están en la capa densa. Por eso, agregar un bloque más (que achica los mapas) puede **reducir** el número de parámetros, mientras que agregar una capa densa oculta lo multiplica.

La tabla muestra el efecto de algunos cambios a `MyCNN`. Todas las convoluciones usan padding $p = (k - 1)/2$.

Variante	Forma antes de aplanar	Parámetros	En capas densas
`MyCNN` (8 y 16 filtros de 3×3)	$16 \times 7 \times 7$	9098	7850
Más filtros (16 y 32)	$32 \times 7 \times 7$	20490	15690
Filtros de 5×5	$16 \times 7 \times 7$	11274	7850

Variante	Forma antes de aplanar	Parámetros	En capas densas
Un tercer bloque (8, 16 y 32 filtros)	$32 \times 3 \times 3$	8778	2890
Una capa densa oculta de 64 unidades	$16 \times 7 \times 7$	52138	50890

Por último, al comparar variantes conviene **cambiar una sola cosa a la vez**. Si se cambian el número de filtros y el tamaño del filtro al mismo tiempo y la exactitud mejora, no hay forma de saber cuál de los dos cambios ayudó. Y la comparación se hace con los datos de validación: los de prueba se usan una sola vez, con el modelo escogido al final.

4. Parte 4: Sobreajuste y regularización

4.1. Las curvas de entrenamiento y validación

Una red con muchos parámetros puede memorizar los datos de entrenamiento, igual que un árbol muy profundo. Se nota en las curvas de la pérdida por época: la de entrenamiento sigue bajando, pero la de validación deja de bajar y empieza a subir. Desde ese momento, la red está aprendiendo detalles de los datos de entrenamiento que no se repiten en datos nuevos.

La Figura 11 lo muestra con una red densa grande (dos capas ocultas de 512 unidades, casi 670 mil parámetros) entrenada con solo 1000 imágenes de MNIST. Para evitar el sobreajuste, se usan más datos, modelos más pequeños, o técnicas de regularización.

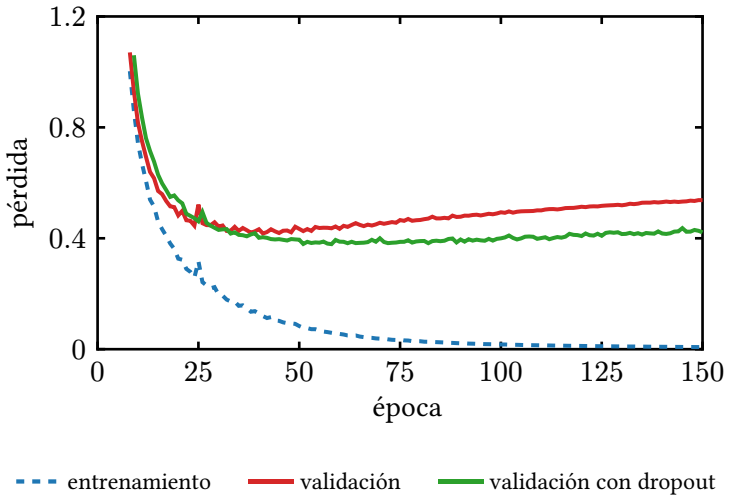


Figura 11: Pérdida de entrenamiento y de validación de una red densa grande entrenada con 1000 imágenes. La pérdida de entrenamiento llega casi a cero, mientras que la de validación baja hasta la época 41 y luego sube: la red sobreajusta. Con dropout (en verde), la pérdida de validación llega más abajo y sube menos.

4.2. Dropout

Dropout es una técnica de regularización específica de las ANN. En cada paso de entrenamiento, cada unidad de una capa se "apaga" (su activación se pone en cero) con probabilidad p, por ejemplo $p = 0.5$, escogiendo al azar unidades distintas en cada paso. Como la red no puede contar con que una unidad en particular esté presente, no puede depender demasiado de ninguna, y aprende features más robustas. Para que la suma de las activaciones no cambie en promedio, las unidades que quedan encendidas se multiplican por $1/(1-p)$.

Al evaluar, en cambio, queremos usar la red completa, sin azar. Aquí es donde importan `model.train()` y `model.eval()`: en modo entrenamiento, `nn.Dropout` apaga unidades; en modo evaluación, no hace nada.

```python
import torch
from torch import nn

torch.manual_seed(0)
dropout = nn.Dropout(p=0.5)
x = torch.ones(1, 8)

dropout.train()          # entrenamiento: apaga unidades al azar
print(dropout(x))
dropout.eval()           # evaluación: no hace nada
print(dropout(x))
```

Salida

```
tensor([[0., 0., 2., 0., 0., 0., 2., 2.]])
tensor([[1., 1., 1., 1., 1., 1., 1., 1.]])
```

Con $p = 0.5$, en modo entrenamiento la mitad de las unidades (en promedio) se apagan y las demás se multiplican por 2. En modo evaluación la entrada pasa sin cambios. En una red, la capa de dropout se declara en `__init__` (`self.dropout = nn.Dropout(0.5)`) y se aplica en `forward`, normalmente después de la ReLU de las capas densas.

4.3. Weight decay y early stopping

Weight decay es regularización L2. En PyTorch se activa con un argumento del optimizador, por ejemplo `torch.optim.SGD(model.parameters(), lr=0.1, weight_decay=1e-4)`.

Esto equivale a sumar a la pérdida una penalización $\frac{\lambda}{2}\|\theta\|^2$ con $\lambda = 10^{-4}$.

Early stopping (parada temprana) consiste en vigilar la pérdida de validación durante el entrenamiento y quedarse con los parámetros de la época en la que fue mínima, en lugar de los de la última época. Para implementarlo, en cada época se compara la pérdida de validación con la mejor hasta ahora y, si mejoró, se guarda una copia de los parámetros, por ejemplo con `torch.save(model.state_dict(), "mejor.pt")`.